

Frost Project — Exact Implementation & Testing Roadmap

We should absolutely not start by implementing the full Frost system.

The document itself is designed around proving the idea in stages and killing bad directions early.

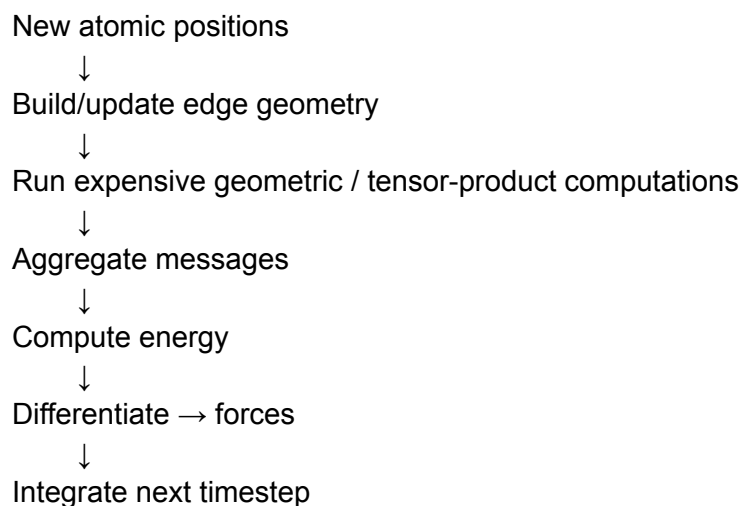
The correct approach is:

Measure → Validate assumptions → Build minimal cache → Prove correctness → Optimize → Compare baselines → Extend.

1. What exactly are we building?

At its core, Frost is an **incremental inference layer for geometric GNN rollouts**.

Normally, at every MD timestep:



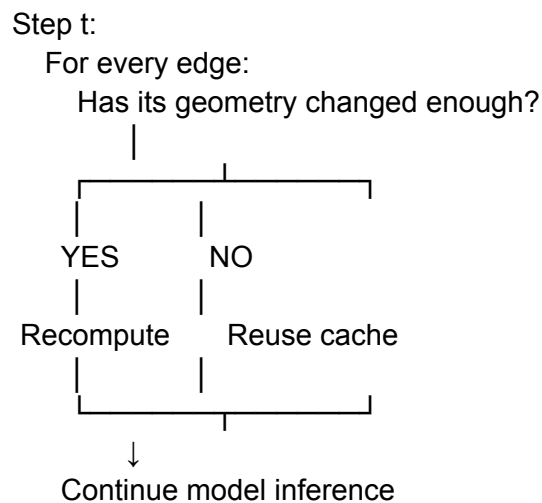
Frost asks:

"Did every edge actually change enough that we need to recompute its expensive geometric feature?"

Instead of recomputing everything:

Step t:
Compute ALL edges

we want:



The important insight in the document is that **we cache edge-local geometric computations, not node states**.

Node caching creates an M-hop dependency explosion. The document argues that edge-level geometric caching avoids this closure problem at the first layer, and for strictly local architectures such as Allegro, the closure can be zero throughout.

2. What should we implement FIRST?

Phase 0 — Do not build Frost yet

This is extremely important.

The first experiment in the document, **E1**, requires essentially a logger, not the entire caching system.

The project should initially answer:

Question 1:

Do edges actually stay similar enough across timesteps?

Question 2:

If we reuse their computation, how much error would that introduce?

The document explicitly makes this the first go/no-go experiment. It uses Allegro and MACE-MP-0 across systems such as silicon, Li3PO4, liquid water, silica glass, and copper, logging per-edge geometric changes and sweeping the tolerance.

3. Phase 1: Build the measurement harness

What we implement

Start with **one model**, preferably:

Choice A: Allegro

Best research choice.

Why?

Because Allegro is strictly local, meaning Frost's edge-local caching mechanism is cleanest there. The document specifically positions Allegro as the first architecture to evaluate because it avoids the deep-layer closure complication.

Choice B: MACE

More widely recognizable and useful for later comparison, but more complicated.

My recommendation

Start:

Allegro

↓

MACE layer 1

↓

MACE deeper layers

↓

Other architectures

The first code should do this

At each timestep:

```

positions_t
positions_t_plus_1

for each edge (i, j):
    r_old = previous displacement
    r_new = current displacement

    delta_r = distance(r_new - r_old)

    log(
        edge_id,
        delta_r,
        atom_species_i,
        atom_species_j,
        timestep
    )

```

Then collect:

delta_r distribution
 delta direction distribution
 fraction of edges below threshold ϵ
 temperature dependence
 system dependence
 spatial clustering of changed edges

The main metric is:

Dirty fraction

$\phi = \frac{\text{number of edges requiring recomputation}}{\text{total number of edges}}$

If:

$$\phi \approx 1$$

then Frost has no opportunity.

If:

$$\phi \approx 0.1$$

then roughly 90% of expensive edge computations could potentially be reused.

The document's first gate says that if the dirty fraction remains too high on favorable systems at a defensible error tolerance, the main mechanism should be stopped rather than spending weeks implementing it.

4. Exact first experiments we should run

Before implementing caching, I would run these **four experiments**.

E1 — Temporal redundancy

Goal

Determine:

How many edges are actually reusable?

Run MD and record:

edge displacement change
edge orientation change
time since previous update
temperature
system type

Output

Plot:

Tolerance ε vs Dirty fraction φ

For example:

ε small $\rightarrow \varphi = 0.90$
 ε medium $\rightarrow \varphi = 0.40$
 ε large $\rightarrow \varphi = 0.10$

But we cannot just choose a huge tolerance. We also need to measure accuracy.

E2 — Cost decomposition

Even if 90% of edges are reusable, Frost is useless if the reusable part is cheap.

We need to profile:

Total model time
└─ Neighbor list

- |— Edge geometry
- |— Tensor products
- |— Message passing
- |— Aggregation
- |— Readout
- |— Backward/gradient

The document defines:

$$\theta = \frac{\text{cacheable computation}}{\text{total computation}}$$

We need to find:

What percentage of inference time is actually removable?

The proposed evaluation instruments tensor-product calls and measures the cacheable share per architecture and layer.

This is **critical**.

A system with:

$$\phi = 0.1$$

$$\theta = 0.1$$

may still provide little speedup.

E3 — Does error attenuate?

This matters later for deeper message-passing architectures.

You intentionally perturb one edge:

Edge perturbation



Layer 1 error



Layer 2 error



Layer 3 error

Measure:

$$\alpha_1, \alpha_2, \alpha_3 \dots \alpha_1, \alpha_2, \alpha_3 \dots$$

Question:

Does a local edge approximation become weaker as it travels through the network?

The document defines this as a key experiment and uses it to determine whether deeper layers can safely exploit attenuated error propagation.

Important

We do not need this to build Frost v0.

This comes after the simple edge cache works.

E4 — Effective closure radius

Measure how far an error actually propagates spatially.

The theoretical receptive field may be:

$M \times \text{cutoff radius}$

But the actual meaningful error region may be much smaller.

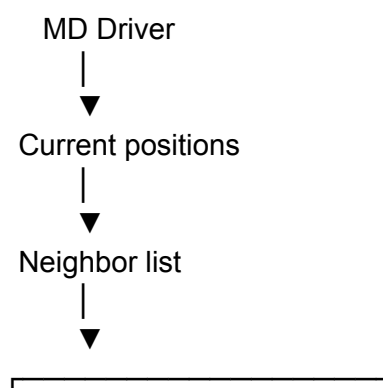
Again, this is research validation before attempting complicated deep-layer reuse.

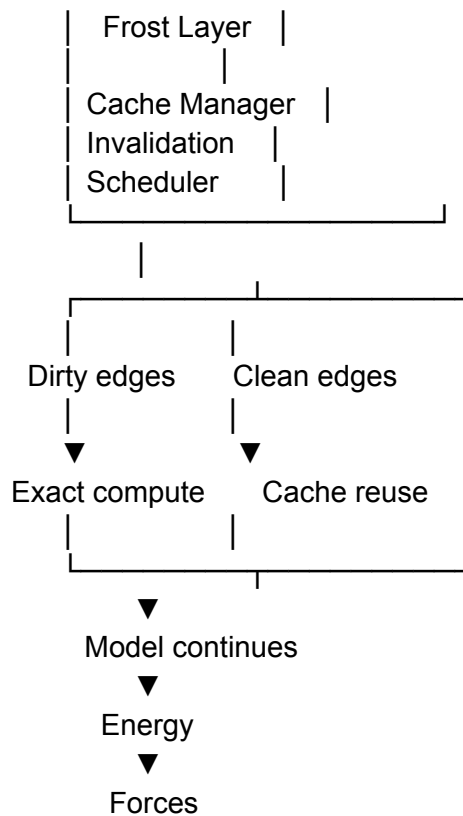
5. Phase 2 — Build Frost-0

Only after E1 and E2 look promising.

Frost-0 should be intentionally simple.

Frost-0 architecture





The document explicitly designs Frost as a wrapper layer between the simulation driver and the model, without modifying model code or changing the driver API.

6. Frost-0 implementation order

Step 1 — Build a Python prototype first

Do **not** immediately write CUDA.

Create something conceptually like:

```
class FrostCache:
    def lookup(edge_id):
        ...

    def insert(edge_id, value):
        ...

    def invalidate(edge_id):
        ...
```

At this stage, performance does not matter.

We are validating:

Can we correctly reuse an edge computation?
Can we detect when it should be recomputed?
Does the model still run?
Do forces remain correct?

Step 2 — Stable edge identity

This is one of the most important engineering problems.

You cannot use:

edge index = 134

because the neighbor list can rebuild.

Instead, the document proposes identifying an edge using the atom pair and periodic boundary image:

(i, j, periodic_image)

and using that as a stable cache key. The cache is designed as a device-resident structure-of-arrays with cached geometry, values, Jacobians, age, error bounds, and keys.

For our prototype:

```
edge_key = (  
    atom_id_i,  
    atom_id_j,  
    periodic_offset  
)
```

Use a Python dictionary first.

Only later move to:

GPU hash table
open addressing
tombstones
compaction

7. Cache state machine

This is a very good place to structure the implementation.

The document specifies states like:

```
EMPTY
↓
PENDING
↓
VALID
↓
DEFERRED
↓
STALE
↓
VALID
```

And when an edge disappears:

```
VALID
↓
TOMBSTONE
↓
EMPTY
```

The document's state machine specifically handles new edges, recomputation, lazy checking, age-based refresh, keyframes, and neighbor-list removals.

For version 1, simplify it:

```
NEW
VALID
DIRTY
REMOVED
```

Then gradually add:

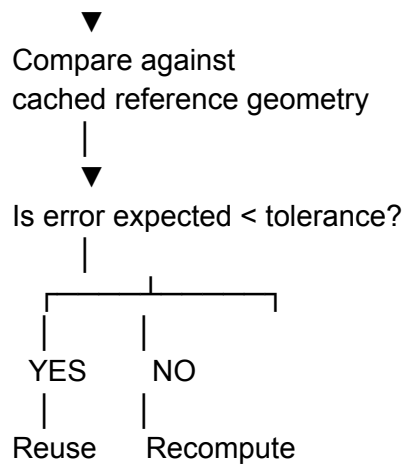
```
DEFERRED
TOMBSTONE
generation counters
```

8. Phase 3 — Invalidation logic

Now we implement the actual Frost decision.

For every edge:

```
current geometry
|
```



The document's invalidation engine evaluates the edge change and uses a compact sensitivity lookup that depends on atom species and binned edge geometry.

Prototype version

Start with something simple:

```

if norm(r_current - r_reference) < threshold:
    reuse()
else:
    recompute()
  
```

Then upgrade to:

```

geometry change
×
estimated sensitivity
×
error propagation factor
<
error budget
  
```

This progression is important.

9. Phase 4 — First-order cache

This is where Frost becomes much more interesting.

A naive cache does:

$$g(r_t) \approx g(r_{\text{cached}})g(r_t) \approx g(r_{\text{cached}})$$

A first-order cache does:

$$g(r_t) \approx g(r_{\text{cached}}) + J(r_{\text{cached}})(r_t - r_{\text{cached}}) \approx g(r_{\text{cached}}) + J(r_{\text{cached}})(r_t - r_{\text{cached}})$$

where:

$$J = \frac{\partial g}{\partial r}$$

So instead of storing only:

cached feature

we store:

cached feature

+

Jacobian

The document's cache layout explicitly includes the cached tensor-product output and its Jacobian, noting that the Jacobian is essential to avoid the force failure of a purely zeroth-order cache.

This is likely a major part of why the projected first-order version can achieve a much lower dirty fraction than zeroth-order reuse.

But:

Do not implement this first.

Order should be:

Exact baseline



Measurement logger



Frost-0 cache



Correctness validation



First-order cache

10. What exactly should we test?

There are three levels of testing.

Level 1 — Unit correctness

Test A: Cache lookup

Insert edge

↓

Lookup same edge

↓

Correct value?

Test B: Edge identity

Neighbor list changes order

↓

Same physical edge

↓

Still gets same cached entry?

Test C: Edge removal

Edge disappears

↓

Cache entry tombstoned/removed

↓

Never reused incorrectly?

Test D: New edge

New edge enters

↓

Must be dirty

↓

Must compute exact value

Level 2 — Differential testing

This is arguably the **most important test**.

Set:

$\epsilon=0$ \epsilon = 0

Then every edge should effectively be recomputed.

Therefore:

Baseline model output

vs

Frost model output

should match, subject to known floating-point/reduction-order differences.

The document explicitly calls this the differential test that should exercise the entire cache, compaction, and dispatch path while removing approximation error, so implementation bugs are isolated from approximation effects. It recommends running this on every commit.

This becomes our CI test.

Every code change:

$\epsilon = 0$

↓

Run baseline

↓

Run Frost

↓

Compare:

energy

forces

outputs

If this fails:

The implementation is broken. Don't continue.

Level 3 — Gradient check

This is extremely important for Frost.

We need to verify:

$$\mathbf{F} = -\nabla E \quad \mathbf{F} = -\nabla E$$

Numerically:

Finite difference:

$$(E(x+h) - E(x-h)) / (2h)$$

versus:

Autograd force

The document specifically identifies this as protection against incorrectly placing the tensor detach, which could silently break the conservative gradient property.

Level 4 — Long trajectory testing

Per-step accuracy isn't enough.

You could have:

tiny error

+

tiny error

+

tiny error

+

...

which eventually causes a bad trajectory.

So run:

Baseline MD

Frost MD

for a long simulation and compare:

energy drift

temperature

structural properties

trajectory divergence

The document recommends a long NVE canary as a guard against accumulated errors that per-step comparisons miss.

11. The experimental roadmap I recommend for us

Week/Stage 1 — Environment

Deliverable

A baseline script that can:

- Load one MLIP
- Run MD
- Build neighbor graph
- Access edge geometry
- Run trajectory

Success criterion

You can reproduce baseline energy/force output reliably.

Stage 2 — Logger

Deliverable

trajectory_logger.py

Logs:

- step
- edge ID
- r_{ij}
- delta r
- delta orientation
- model output/error proxy

Run

Multiple systems and temperatures.

Output

- dirty fraction curves
- histograms
- edge-change distributions

Decision

This is our **first actual research result**.

Stage 3 — Profile

Run:

Nsight
PyTorch profiler
CUDA events

Measure:

cacheable fraction θ
tensor-product cost
aggregation cost
overhead budget

Now we know:

Is there enough redundant expensive work to remove?

Stage 4 — Python Frost-0

Implement:

Stable edge key
Cache
Simple invalidation
Reuse/recompute

No CUDA optimization yet.

Goal

Correctness only.

Stage 5 — Differential validation

Run:

$\varepsilon = 0$

Compare:

baseline vs Frost

Test every:

energy
force
trajectory step
neighbor rebuild

Stage 6 — Approximation experiments

Sweep:

$\epsilon = 10^{-x}$...

Measure:

dirty fraction
force error
energy error
speed

Generate:

Speedup vs accuracy curve

Stage 7 — Fixed-stride baseline

This is absolutely necessary.

Compare Frost against:

Baseline
Fixed stride = 2
Fixed stride = 4
Fixed stride = 8
Frost adaptive

Why?

Because if:

simple fixed stride
 $\approx$
Frost

then the adaptive mechanism is not providing a strong contribution.

The document explicitly treats this as a decisive comparison and places it as a major decision gate.

12. Only after that: CUDA optimization

Once the algorithm works:

Optimize in this order

1. Fused predicate + gather

Instead of:

Kernel 1: check edge
Kernel 2: create mask
Kernel 3: compact
Kernel 4: gather
Kernel 5: compute

move toward:

One kernel:
 traverse edge
 check validity
 directly compact dirty edges

The document specifically proposes fusing predicate evaluation and gathering to avoid multiple reads of the edge data.

2. Sparse tensor-product dispatch

Only compute:

dirty edges

instead of:

all edges

3. Scatter results back

cached values

+
freshly recomputed values
↓
reconstruct full edge feature set
↓
aggregation

4. CUDA graphs

Only after dynamic execution works.

5. Multi-GPU

Very late.

Don't touch this until the single-GPU version is correct and benchmarked.

13. What I think our actual MVP should be

Forget the full 4,000+ LoC system initially.

Our first serious Frost prototype should be:

FROST MVP

Model:

Allegro OR MACE

Driver:

ASE

Cache:

Python / PyTorch dictionary or tensor structure

Cached item:

Edge-local geometric feature

Invalidation:

$||r_{\text{current}} - r_{\text{cached}}|| < \epsilon$

Reuse:

cached feature

Refresh:

recompute dirty edge

Validation:

$\varepsilon = 0$ differential test

force comparison

finite-difference gradient check

short trajectory

That is enough to answer:

Does the core Frost mechanism work at all?

14. Then the research-quality version becomes

Frost MVP



Stable edge identity



Neighbor-list rebuild handling



Adaptive error predicate



First-order Taylor cache



Error attenuation



Refresh scheduler



Canary checks



Fallback to baseline



CUDA optimization



Multi-model evaluation



Multi-GPU

The document's full architecture contains a cache manager, invalidation engine, and refresh scheduler as the main Frost layer, while preserving existing model and driver implementations.

15. My strongest recommendation

We should not begin by implementing the architecture shown in the full paper.

The document itself gives us the correct research strategy:

First milestone:

Prove redundancy exists.

Second:

Prove the expensive part is cacheable.

Third:

Build the simplest possible edge cache.

Fourth:

Prove $\epsilon = 0$ matches the baseline.

Fifth:

Show a real speedup–accuracy tradeoff.

Sixth:

Beat fixed-stride caching.

Only after that should we spend serious effort on:

Taylor cache
attenuation
CUDA fusion
CUDA graphs
multi-GPU
LAMMPS production integration

The document is unusually explicit that the project should fail early if its central assumptions fail, with a first decision gate around temporal redundancy and a second around whether adaptive caching actually beats simple fixed-stride reuse.